C.3.3. AI Scientist Team Code Review

Domain Adaptation and Multi-dataset training

The paper seems to describe the "Domain Adaptation" experiment as primarily focused on transferring ImageNet-pretrained models to other vision datasets. After reviewing the code, we found attempts to implement a domain adaptation technique by training a separate classifier to distinguish different domains, but these attempts were unsuccessful. In the end, the AI Scientist opted for an implementation that does not include this domain adaptation technique.

Moreover, in the code where this domain adaptation technique was implemented, multi-dataset training was correctly performed as well–training a single model on all three datasets with domain discriminator loss, as shown in fig. 11. Had this code run successfully, the AI Scientist would likely have chosen it over the one ultimately selected, which lacked proper multi-dataset training but ran without errors.

```python
class DomainDiscriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(FEATURE_DIM, 256)
        self.fc2 = nn.Linear(256, 3)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        return self.fc2(x)
```

```python
# Training loop
for epoch in range(NUM_EPOCHS):
    feature_extractor.train()
    domain_discriminator.train()
    classifier.train()

    for dataset_name, (train_dataset, test_dataset) in datasets.items():
        train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)

        for batch in train_loader:
            # Move batch to device
            images = batch["image"].to(device)
            labels = batch["label"].to(device)

            # Feature extraction
            features = feature_extractor(images)

            # Classification loss
            clf_outputs = classifier(features)
            clf_loss = F.cross_entropy(clf_outputs, labels)

            # Domain adversarial loss
            domain_outputs = domain_discriminator(features)
            domain_labels = torch.zeros(images.size(0), dtype=torch.long).to(device)
            domain_loss = F.cross_entropy(domain_outputs, domain_labels)

            # Total loss
            total_loss = clf_loss - 0.1 * domain_loss

            # Optimization
            fe_optimizer.zero_grad()
            clf_optimizer.zero_grad()
            disc_optimizer.zero_grad()
            total_loss.backward()
            fe_optimizer.step()
            clf_optimizer.step()
            disc_optimizer.step()
```

Figure 11 | Domain discriminator and multi-dataset training loop.

Environmental noise implementation

The paper states, "To simulate challenging environmental conditions, we applied data augmentations during testing, including brightness and contrast adjustments, Gaussian blur, and random affine transformations." This is confirmed in the code, as shown in fig. 12.

The calculation of the Environmental Robustness Score–a metric introduced by the AI Scientist and defined as "the ratio of model accuracy under challenging conditions to that under normal conditions, to quantify robustness"–matches the description in the paper, as shown in fig. 13.